
SESSION 9

MCP: one protocol, many clients

How agents plug into tools, how the bytes move, and one server driven by two different clients.

Today's plan

01 MCP concepts

Architecture, server primitives, client primitives

02 Transports

stdio, streamable HTTP, and the bridge pattern

03 Live demo

Playwright MCP in Claude Code, then in a LangChain agent

04 Into the runtime

Context, state, messages, human in the loop, multi-agent

Why MCP: integrations stop multiplying

WITHOUT A STANDARD

$$N \times M$$

Every app writes a custom connector for every tool. 5 apps and 10 tools means 50 integrations.

WITH MCP

$$N + M$$

Each app speaks MCP once, each tool ships one server. 5 + 10 = 15 pieces.

Think USB-C for AI apps: **the protocol is the interface, the client is interchangeable.**

Three roles: host, client, server

Host

The AI app the user sees: Claude Code, Claude Desktop, your LangChain agent.

MCP client

Lives inside the host. One per server connection.



MCP server

Wraps a real system and exposes it.

Playwright → browser

GitHub → repos

Postgres → tables

One host can hold many clients, one for each server it connects to.

Server primitives: what a server offers

The model decides

Tools

Functions the agent can call.

Playwright: navigate, click, snapshot

The app decides

Resources

Data pulled in as context.

A file, a table schema, a page

The user decides

Prompts

Reusable templates.

Often shown as slash commands

Most servers in the wild ship only tools.

THE REVERSE DIRECTION

Client primitives: the server asks back

Roots

The client declares a boundary before anything runs.

Imagine: "work only inside /home/you/project"

Sampling

The server borrows the client's LLM. No API key of its own.

Imagine: Playwright grabs the page, asks your model which story matters

Elicitation

The server pauses and asks the user a structured question.

"Which repository did you mean?" · added June 2025

Playwright MCP uses none of these. They are the underused half of the protocol.

Two transports: where does the server live?

	stdio	Streamable HTTP
Runs as	Subprocess on your machine	A web endpoint, anywhere
Talks over	stdin / stdout	POST to one URL
Lifecycle	Client starts it, stops it on disconnect	Already running; you just connect
Users	One: you	Many at once
Auth	None, inherits your machine	OAuth 2.1, scoped tokens
Good for	Browser, files, desktop apps	Hosted services like GitHub

Rule of thumb: **touches your machine → stdio. Shared or hosted → HTTP.**

A stdio config is just a terminal command

CLIENT CONFIG

```
"transport": "stdio", "command": "uv", "args":  
["run", "python", "-m", "mcp_server_time", "--  
local-timezone=Europe/London"]
```

SAME THING, BY HAND

```
$ uv run python -m mcp_server_time \ --local-  
timezone=Europe/London  
# ...and it just sits there, # waiting for JSON-RPC  
on stdin
```

No shell

Args are a list. No pipes, wildcards, && or \$VARS; pass env explicitly.

Client owns it

Spawned on connect, kept for the session, stopped on disconnect.

Mind the directory

uv finds the project from where it starts. Add --directory or use absolute paths.

Streamable HTTP in four ideas

- 1 **One endpoint.** Every client message is a POST.
- 2 **Server picks the reply:** plain JSON, or an event stream for slow work.
- 3 **Optional session ID** in the Mcp-Session-Id header lets a stateful server remember you.
- 4 **Resumable.** A dropped stream reconnects from the last event seen.

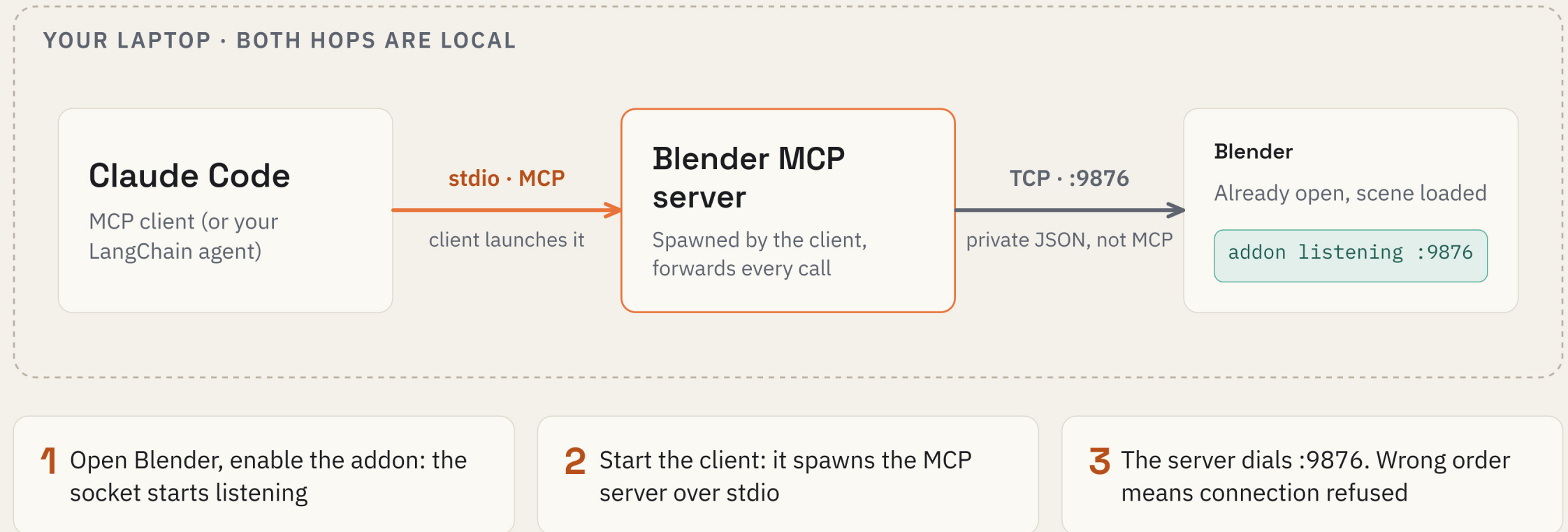
JAVA ANALOGY

JSON reply $\approx$ **Mono**

Event stream $\approx$ **Flux**

But it is a wire format, not Reactor: no backpressure. It replaced the old HTTP+SSE transport, which needed two endpoints.

Already-running apps: the bridge pattern



DEMO 1

Playwright MCP in Claude Code

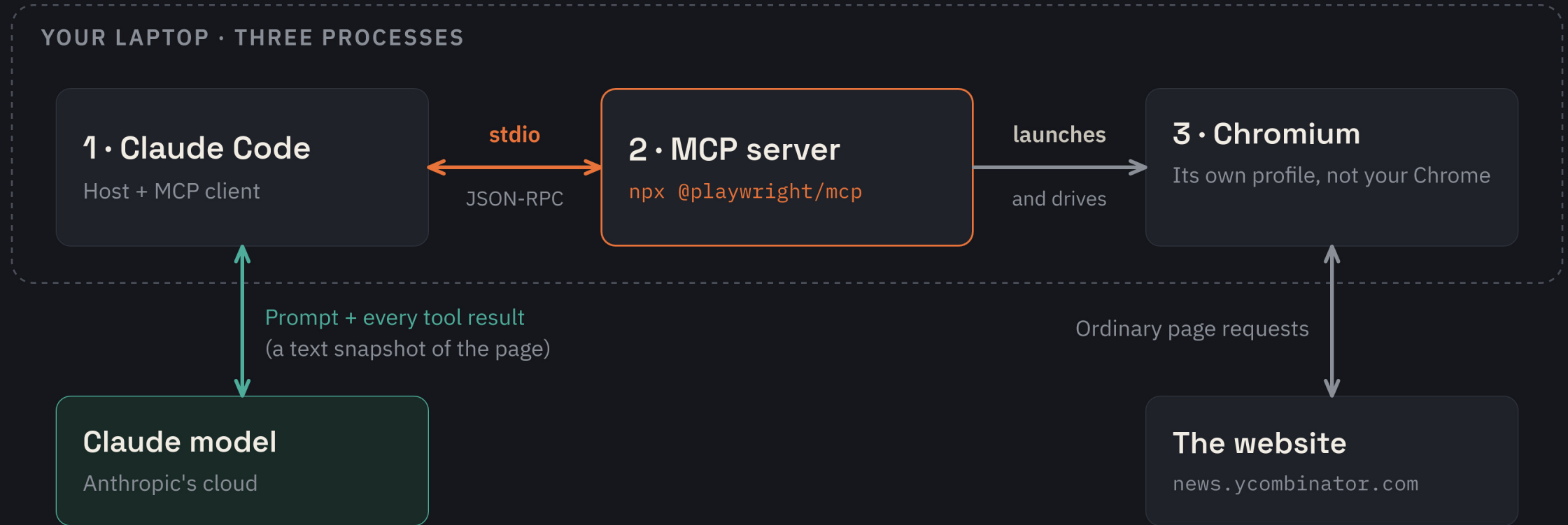
```
$ claude mcp add playwright -- npx @playwright/mcp@latest  
> /mcp # confirm it is connected  
  
> use the playwright mcp to open news.ycombinator.com  
and give me the top 3 titles
```



Watch the screen: a real browser opens and navigates. It is visible by default.

DEMO 1 · UNDER THE HOOD

Processes are local. Thinking is not.



DEMO 2

Same server, driven by LangChain

```
client = MultiServerMCPClient({"playwright": {
    "transport": "stdio",
    "command": "npx",
    "args": ["@playwright/mcp@latest"],
}})

async with client.session("playwright") as s:
    tools = await load_mcp_tools(s)
    agent = create_agent(model, tools)
    await agent.ainvoke({"messages": question})
```

The gotcha

The client is stateless by default: every tool call opens a fresh session. For Playwright, a new browser per call.

`client.session()` keeps one browser alive:
your first taste of state.

**Same server.
Two clients.
Zero server changes.**

The protocol is the interface. The client is interchangeable.

Trust models: a preview for next time

LOCAL · STUDIO

Trust the process

No OAuth. The server inherits whatever your machine and browser already have.

Playwright drives a browser that is logged in as you.

REMOTE · HTTP

Trust the token

The server is an OAuth 2.1 resource server: it validates tokens, never issues them.

RFC 8707 binds each token to one server, so a stolen token is useless elsewhere.

Next session: GitHub's remote MCP server. Same prompt, read-only token vs write token.

SECOND HALF

Into the runtime

Runtime context and state	Why <code>client.session()</code> mattered
Managing messages	Keeping the conversation history under control
Human in the loop	Pausing for approval. Elicitation is MCP's version
Multi-agent	Agents handing work to agents
Dynamic models	If time allows

MANAGING MESSAGES

Role strings → message classes

Role string	Message class
"user" or "human"	HumanMessage
"assistant" or "ai"	AIMessage
"system"	SystemMessage
"tool"	ToolMessage needs tool_call_id

Pass a **string** or a **class**; LangChain turns both into the same message object.